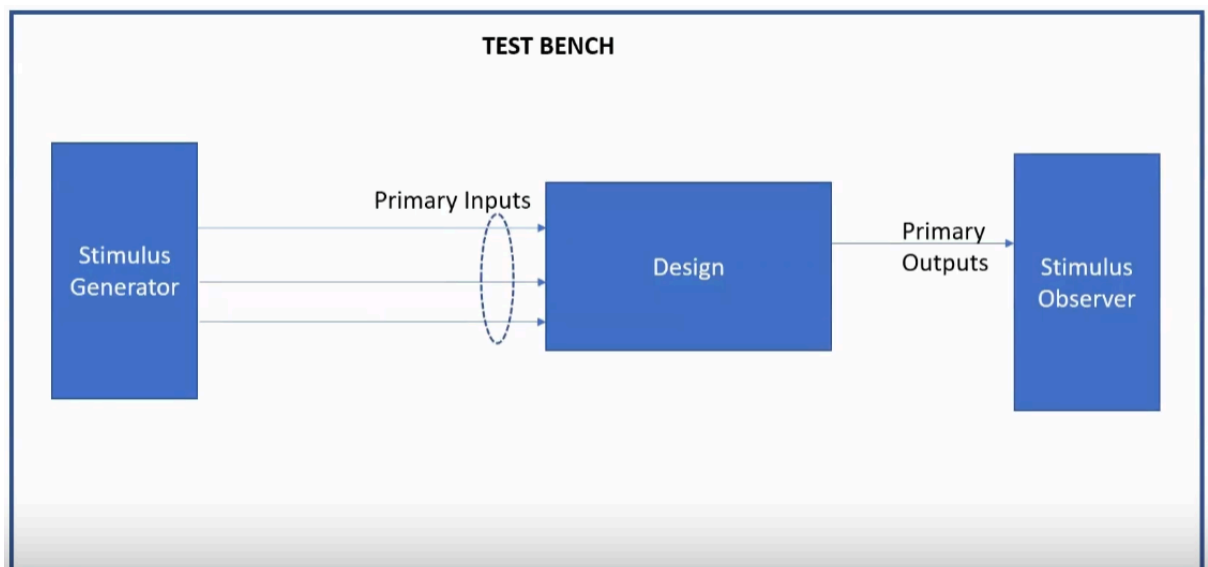


“Week 1 Notes”

Week 1 Notes

“Week 1 Notes”

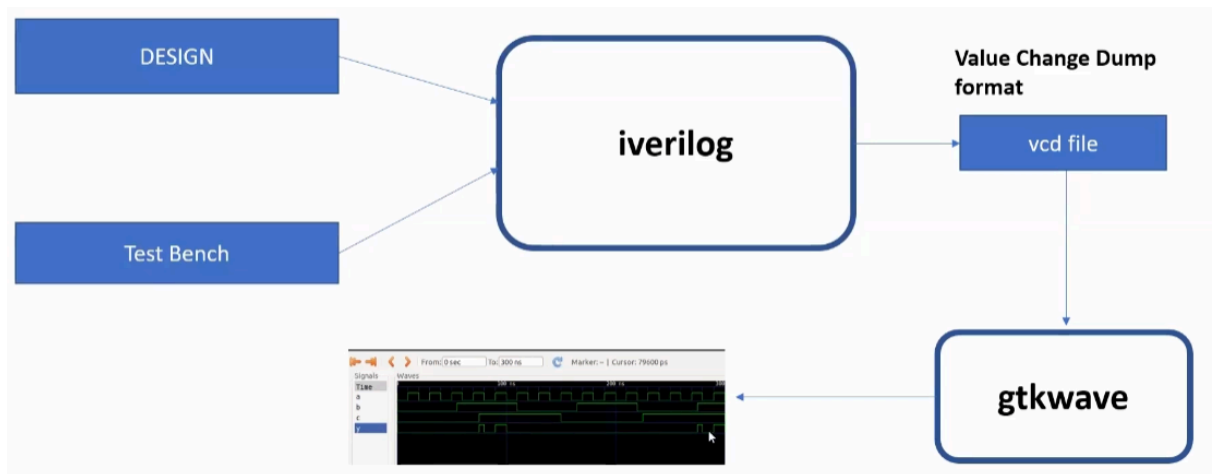
1. **Simulator** - is a tool!
 - a. Used to checking my design.
 - b. RTL design is the implementation of the specification.
 - c. Intent of the specification should be verified by simulating the design.
 - d. “Iverilog” - for simulating.
 - e. **Working:**
 - i. Look for changes in input signals.
 - ii. Upon every change in input values, output values are evaluated.
2. **RTL Design** - derived from specification - Static in nature!
 - a. Verilog code file or files in .v extensions.
 - b. Written to mimic the desired functionality documented in the specification.
 - c. With some changes based on the achievability of a certain functionality.
3. **Testbench** - Verify the Design - Dynamic in nature!
 - a. Applies test vectors to my design.
 - b. Check the output and match it with the specification.
4. **Design and Testbench** - setup! - [N(in) and N(out) is based on spec]!



- a. Primary inputs.
 - i. Input ports of the design - [N in number].
 - ii. Used to drive input stimulus to the design.
- b. Primary outputs.
 - i. To be captured and compared based on the specification expected values or behaviour - [Output ports] - [N in number].
- c. **Stimulus generator and driver.**
 - i. Generates input stimulus and drives it to the design.
- d. **Stimulus Observer.**
 - i. Capture the output values.
 - ii. Compared with the expected values from spec.

“Week 1 Notes”

5. iverilog simulation flow and gtk-wave!



- a. Design & Testbench is compiled using “iverilog”.
 - i. Compile -> Elaboration -> Simulation
- b. **IN** stim is driven and **OP** response is captured.
- c. A “**.vcd**” - value change dump file is created.
 - i. Using “\$dumpfile & \$dumpvars” - written in TB.
 - ii. To observe the signalling of our design and debug any abnormalities.
- d. Design and Testbench are connected via “**interface**” or in this case **module instantiation**.

“Week 1 Notes”

6. mkdir and git clone - [Lab 1]

- mkdir vsd -> cd vsd -> mkdir VLSI
- git clone "sky130".

```
w0rbl33@w0rbl33-VirtualBox: ~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop
928663 -rw-rw-r-- 1 w0rbl33 w0rbl33 0 Sep 19 17:37 .magic_tkcon_hst
917515 drwx----- 5 w0rbl33 w0rbl33 4096 Sep 19 18:00 snap/
917880 drwxrwxr-x 17 w0rbl33 w0rbl33 4096 Sep 19 18:36 yosys/
917552 drwx----- 12 w0rbl33 w0rbl33 4096 Sep 19 18:37 .config/
917573 -rw----- 1 w0rbl33 w0rbl33 1458 Sep 19 19:27 .bash_history
917854 drwxrwxr-x 2 w0rbl33 w0rbl33 4096 Sep 21 11:45 vsd/
917506 drwxr-x-- 18 w0rbl33 w0rbl33 4096 Sep 21 11:45 /

w0rbl33@w0rbl33-VirtualBox:~$ cd vsd/
w0rbl33@w0rbl33-VirtualBox:~/vsd$ mkdir L1
w0rbl33@w0rbl33-VirtualBox:~/vsd$ cd L1/
w0rbl33@w0rbl33-VirtualBox:~/vsd/L1$ cd ..
w0rbl33@w0rbl33-VirtualBox:~/vsd$ mkdir VLSI
w0rbl33@w0rbl33-VirtualBox:~/vsd$ cd VLSI/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ mkdir vsdflow
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ git clone https://github.com/kunalg123/sky130RTLDesignAndSynthesisWorkshop.git
Cloning into 'sky130RTLDesignAndSynthesisWorkshop'...
remote: Enumerating objects: 417, done.
remote: Counting objects: 100% (69/69), done.
remote: Compressing objects: 100% (52/52), done.
remote: Total 417 (delta 19), reused 47 (delta 12), pack-reused 348 (from 1)
Receiving objects: 100% (417/417), 7.79 MiB | 8.68 MiB/s, done.
Resolving deltas: 100% (242/242), done.
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ cd sky130RTLDesignAndSynthesisWorkshop/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop$ pwd
/home/w0rbl33/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop$
```

7. Working with “iverilog and gtk-wave”.

- Navigate to “verilog_files” in the above cloned repo.
- Loading design and testbench in simulator.
 - cmd - “iverilog “design_name” “tb_name” ”.
- It generates an “a.out” file.
- Execute this “a.out” file.
 - cmd “. /a.out ”.
 - It will dump a “.vcd” file - waveform!
- Load this file in gtk-wave.
 - cmd “gtkwave “tb_name.vcd” ”

```
w0rbl33@w0rbl33-VirtualBox: ~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files
Cloning into 'sky130RTLDesignAndSynthesisWorkshop'...
remote: Enumerating objects: 417, done.
remote: Counting objects: 100% (69/69), done.
remote: Compressing objects: 100% (52/52), done.
remote: Total 417 (delta 19), reused 47 (delta 12), pack-reused 348 (from 1)
Receiving objects: 100% (417/417), 7.79 MiB | 8.68 MiB/s, done.
Resolving deltas: 100% (242/242), done.
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ cd
sky130RTLDesignAndSynthesisWorkshop/ vsdflow/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI$ cd sky130RTLDesignAndSynthesisWorkshop/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop$ pwd
/home/w0rbl33/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop$ cd verilog_files/
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ iverilog good_mux.v tb_good_
tb_good_counter.v tb_good_latch.v tb_good_mux.v tb_good_shift_reg.v
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ iverilog good_mux.v tb_good_mux
.v
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ ./a.out
VCD info: dumpfile tb_good_mux.vcd opened for output.
tb_good_mux.v:23: $finish called at 300000 (1ps)
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ gtkwave tb_good_mux.vcd

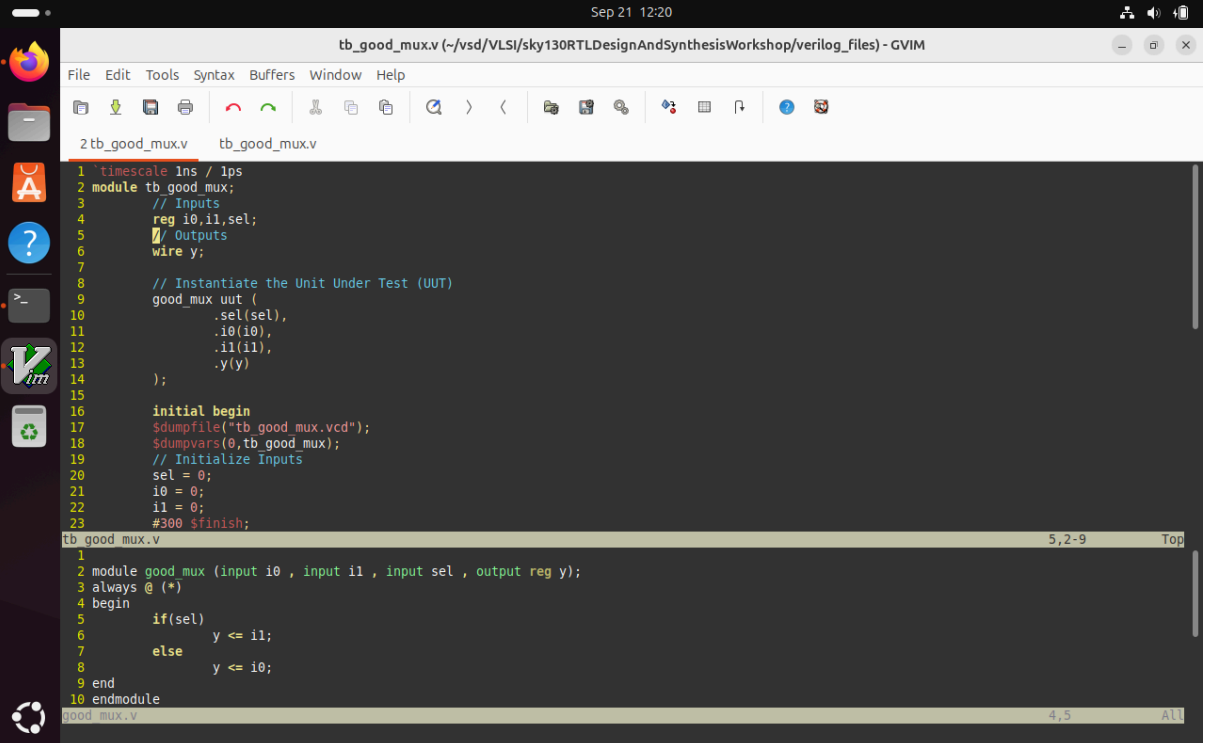
GTKWave Analyzer v3.3.116 (w)1999-2023 BSI

[0] start time.
[300000] end time.
Exiting.
w0rbl33@w0rbl33-VirtualBox:~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files$
```

“Week 1 Notes”

8. Text editor - gVim

- a. cmd - “ gvim “file_name.ext” “



The screenshot shows the gVim text editor interface. The title bar indicates the file path: `tb_good_mux.v (~/.vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files) - GVIM`. The menu bar includes File, Edit, Tools, Syntax, Buffers, Window, and Help. The toolbar contains various icons for file operations and editing. The main editor window displays two files: `2 tb_good_mux.v` and `tb_good_mux.v`. The code is as follows:

```
1 timescale 1ns / 1ps
2 module tb_good_mux;
3     // Inputs
4     reg i0, i1, sel;
5     // Outputs
6     wire y;
7
8     // Instantiate the Unit Under Test (UUT)
9     good_mux uut (
10         .sel(sel),
11         .i0(i0),
12         .i1(i1),
13         .y(y)
14     );
15
16     initial begin
17         $dumpfile("tb_good_mux.vcd");
18         $dumpvars(0, tb_good_mux);
19         // Initialize Inputs
20         sel = 0;
21         i0 = 0;
22         i1 = 0;
23         #300 $finish;
24 endmodule
```

The bottom status bar shows the current line and column: `5,2-9 Top` for the first file and `4,5 All` for the second file.

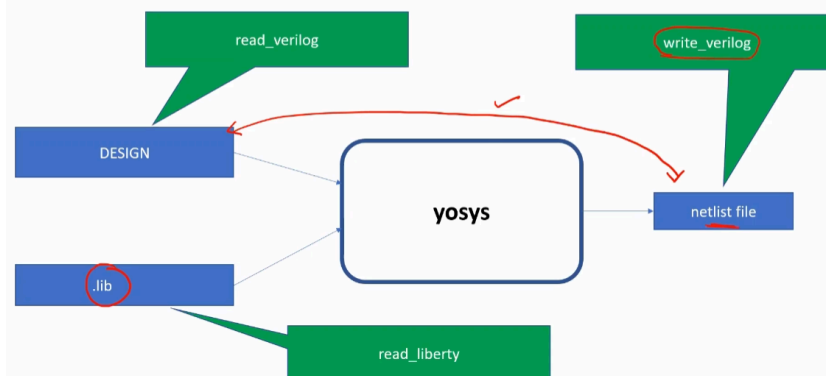
“Week 1 Notes”

9. Working with “syosys”.

a. Synthesizer!

- i. Tool used to convert RTL -> netlist.
- ii. read_verilog command to read the “rtl-design”.
- iii. read_liberty command to read the “.lib files”.
- iv. write_verilog command to generate “netlist - [true representation of the desing]”.
 1. Representation of the rtl design in the form of standard cells present in “.lib”.

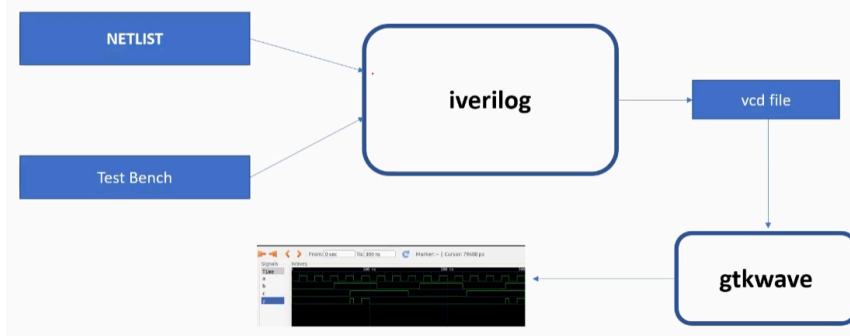
Yosys setup



b. Verify synthesizer's output!

- i. Debug the design after synthesis.
- ii. “netlist” + “testbench” -> “simulator “iVerilog” ” - simulate.
- iii. “.vcd” file obtained -> gtkwave “file_name.vcd”. - observer stimulus.
- iv. This Stimulus should be the same as the stimulus obtained during RTL design testing.
 1. Input and output ports do not change.

Verify the synthesis

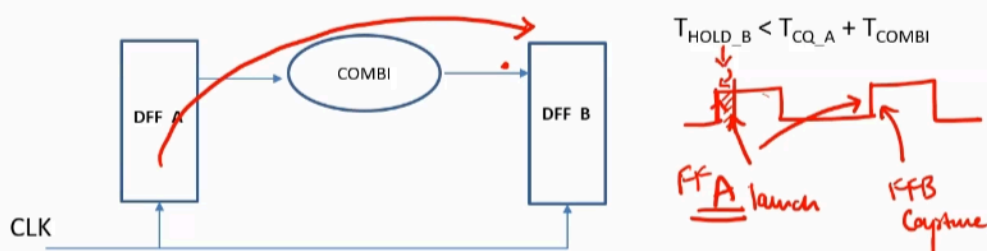


“Week 1 Notes”

c. Logic Synthesis!

- i. RTL Design
 1. Spec written in behavioural form in HDL language.
- ii. Synthesis
 1. RTL to Gate level translation.
 2. Connections are made.
 3. Given out as a "netlist".
- iii. “.lib”
 1. Collection of standard cells - logic gates.
 2. Different flavours of same gates:
 - a. Slow
 - b. Medium
 - c. Fast speed.
 3. Rich enough to implement any functionality.
- iv. Why different flavour gates!
 1. Combinational delay in the logic path determines the maximum speed of operation of a digital logic circuit.
 - a. Based on clk freq.
 - b. $T_{clk} > T_{cq_a}[\text{propagation delay of FL_A}] + T_{combi}[\text{propagation delay of comb ckt}] + T_{setup_b}[\text{duration before which the data should be stable}]$
 - i. T_{clk} = time for data to travel from flop A to flop B is achieved in one clock cycle.
 - c. Determine the max clk freq = $1/T_{clk}$.
 - d. Cells should work fast to make T_{combi} small.

Why we need slow cells ?

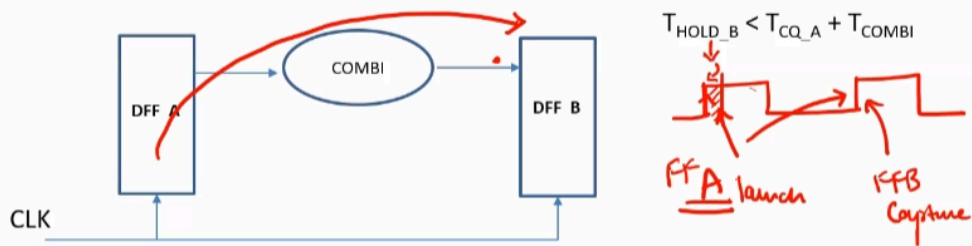


- To ensure that there are no “HOLD” issues at DFF_B , we need cells that work slowly 😊 !!!
- Hence we need cells that work fast to meet the required performance and we need cells that work slow to meet HOLD
- The collection forms the .lib

“Week 1 Notes”

- v. Why do we need slow cells!
1. To avoid hold time violations.
 2. $T_{hold_b} < T_{cq_a} + T_{combi}$.
 3. Delay from A to B should be greater than the hold time.
 4. Hold time - time period during which the data of Flop should not change after the clock edge.
 5. Cells should be fast to meet setup requirements and slow to meet hold requirements.

Why we need slow cells ?



- To ensure that there are no “HOLD” issues at DFF_B , we need cells that work slowly 😊 !!!
- Hence we need cells that work fast to meet the required performance and we need cells that work slow to meet HOLD
- The collection forms the .lib

“Week 1 Notes”

- vi. Fast cells v Slow Cells
 1. The capacitor is the load in digital ckt.
 2. Propagation delay
 - a. Time for input to cause a change in output.
 3. Current carrying capability of a transistor - is the function of its “width”.
 4. Cannot compromise on the Area and power.

Faster Cells vs Slower Cells

- Load in Digital Logic circuit → Capacitance
- Faster the charging / discharging of capacitance → Lesser the cell delay
 - To charge / discharge the capacitance fast , we need transistors capable of sourcing more current
 - Wider transistors -> Low Delay -> More Area and Power as well !!
 - Narrow transistors -> More Delay -> Less Area and Power
 - Faster cells donot come free , they come at penalty of area and power

“Week 1 Notes”

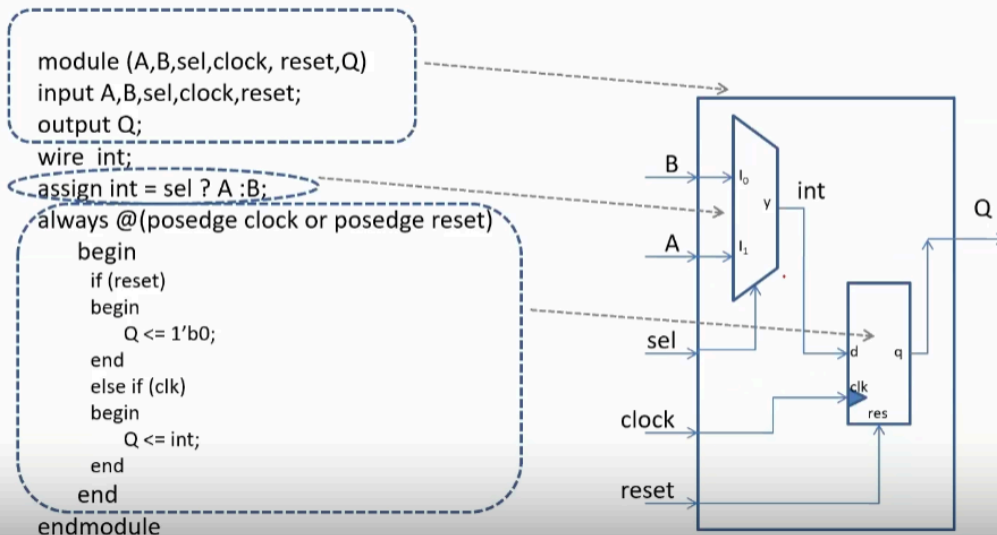
vii. Selection of cells

Selection of Cells

- Need to guide the Synthesizer to select the flavour of cells that is optimum for the implementation of logic circuit
- More use of faster cells
 - Bad circuit interms of Power and Area
 - Hold time violations ??
- More use of slower cells
 - Sluggish circuit , may not meet the performance need
- The guidance offered to the Synthesizer → “Constraints”

1. Syntactical check
2. Mapping the design to digital ckt.
3. Netlist is obtained.

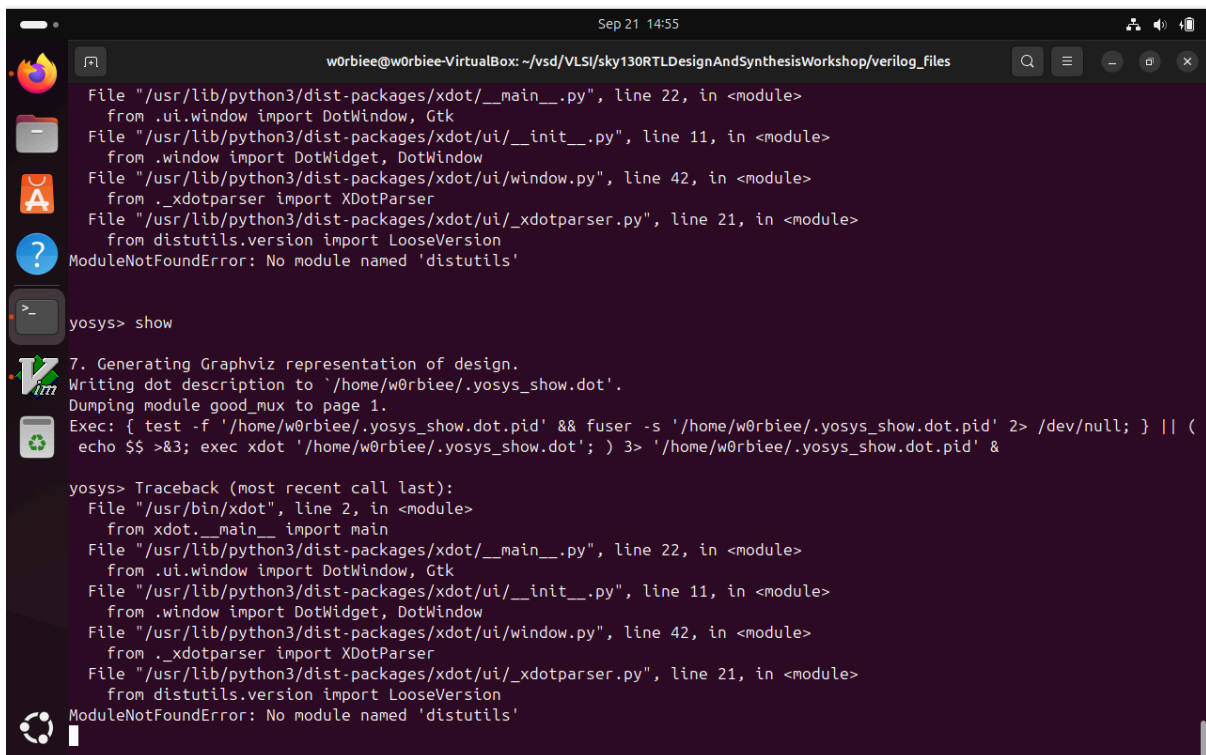
Synthesis (Illustration)



“Week 1 Notes”

10. Yosys working! - LAB3

- a. Invoke “yosys”
 - i. cmd - yosys
 - ii. cmd - read_liberty -lib “library’s path”
 1. Lib used - “sky130_fd_sc_hd_tt_025C_1v80.lib”
 - iii. cmd - read_verilog “design_name”
 1. Here using “good_mux.v”
 - iv. synth -top “design module”
 1. Generated from step “iii”.
 - v. cmd - abc -liberty “lib path”
 1. To generate netlist.
 2. RTL file to gates.
 - a. Gate selection is specified in the library.
 - vi. cmd - show
 1. To see the graphical version of the logic.
 2. **[ERROR encountered]**
 - a. **Xdot not found**
 - i. **Fixed with importing libraries**



```
Sep 21 14:55
w0rbi3e@w0rbi3e-VirtualBox: ~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files

File "/usr/lib/python3/dist-packages/xdot/__main__.py", line 22, in <module>
    from .ui.window import DotWindow, Gtk
File "/usr/lib/python3/dist-packages/xdot/ui/__init__.py", line 11, in <module>
    from .window import DotWidget, DotWindow
File "/usr/lib/python3/dist-packages/xdot/ui/window.py", line 42, in <module>
    from ..xdotparser import XDotParser
File "/usr/lib/python3/dist-packages/xdot/ui/_xdotparser.py", line 21, in <module>
    from distutils.version import LooseVersion
ModuleNotFoundError: No module named 'distutils'

yosys> show

7. Generating Graphviz representation of design.
Writing dot description to '/home/w0rbi3e/.yosys_show.dot'.
Dumping module good_mux to page 1.
Exec: { test -f '/home/w0rbi3e/.yosys_show.dot.pid' && fuser -s '/home/w0rbi3e/.yosys_show.dot.pid' 2> /dev/null; } || (
echo $$ >&3; exec xdot '/home/w0rbi3e/.yosys_show.dot'; ) 3> '/home/w0rbi3e/.yosys_show.dot.pid' &

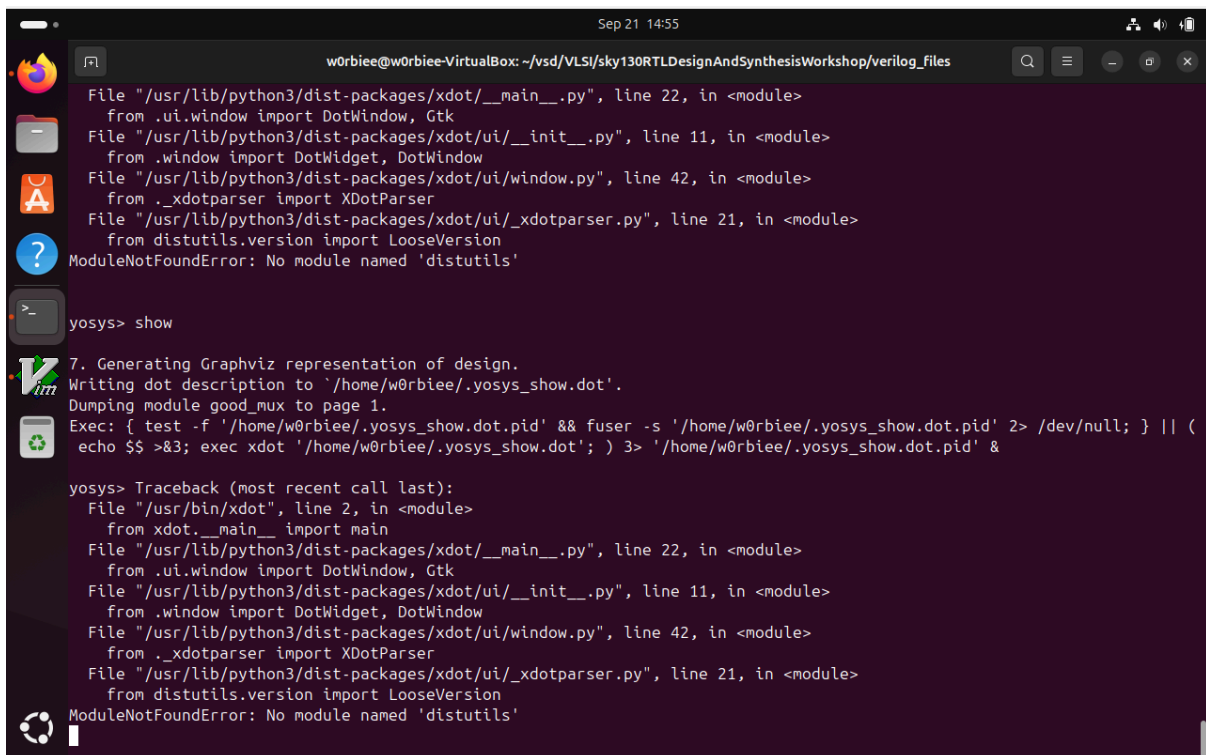
yosys> Traceback (most recent call last):
  File "/usr/bin/xdot", line 2, in <module>
    from xdot.__main__ import main
  File "/usr/lib/python3/dist-packages/xdot/__main__.py", line 22, in <module>
    from .ui.window import DotWindow, Gtk
  File "/usr/lib/python3/dist-packages/xdot/ui/__init__.py", line 11, in <module>
    from .window import DotWidget, DotWindow
  File "/usr/lib/python3/dist-packages/xdot/ui/window.py", line 42, in <module>
    from ..xdotparser import XDotParser
  File "/usr/lib/python3/dist-packages/xdot/ui/_xdotparser.py", line 21, in <module>
    from distutils.version import LooseVersion
ModuleNotFoundError: No module named 'distutils'
```

- vii. Cmd - write_verilog “filename.v”
 1. Creates a netlist file.
 2. use -noattr - gives a cleaner netlist.
- b. Synthesize the design.

“Week 1 Notes”

1. Yosys working! - LAB3

- a. Invoke “yosys”
 - i. cmd - yosys
 - ii. cmd - read_liberty -lib “library’s path”
 1. Lib used - “sky130_fd_sc_hd_tt_025C_1v80.lib”
 - iii. cmd - read_verilog “design_name”
 1. Here using “good_mux.v”
 - iv. synth -top “design module”
 1. Generated from step “iii”.
 - v. cmd - abc -liberty “lib path”
 1. To generate netlist.
 2. RTL file to gates.
 - a. Gate selection is specified in the library.
 - vi. cmd - show
 1. To see the graphical version of the logic.
2. [ERROR encountered]
 - a. Xdot not found
 - i. Fixed with importing libraries



```
w0rbiee@w0rbiee-VirtualBox: ~/vsd/VLSI/sky130RTLDesignAndSynthesisWorkshop/verilog_files
File "/usr/lib/python3/dist-packages/xdot/__main__.py", line 22, in <module>
    from .ui.window import DotWindow, Gtk
File "/usr/lib/python3/dist-packages/xdot/ui/__init__.py", line 11, in <module>
    from .window import DotWidget, DotWindow
File "/usr/lib/python3/dist-packages/xdot/ui/window.py", line 42, in <module>
    from ..xdotparser import XDotParser
File "/usr/lib/python3/dist-packages/xdot/ui/_xdotparser.py", line 21, in <module>
    from distutils.version import LooseVersion
ModuleNotFoundError: No module named 'distutils'

yosys> show

7. Generating Graphviz representation of design.
Writing dot description to '/home/w0rbiee/.yosys_show.dot'.
Dumping module good_mux to page 1.
Exec: { test -f '/home/w0rbiee/.yosys_show.dot.pid' && fuser -s '/home/w0rbiee/.yosys_show.dot.pid' 2> /dev/null; } || (
echo $$ >&3; exec xdot '/home/w0rbiee/.yosys_show.dot'; ) 3> '/home/w0rbiee/.yosys_show.dot.pid' &

yosys> Traceback (most recent call last):
  File "/usr/bin/xdot", line 2, in <module>
    from xdot.__main__ import main
  File "/usr/lib/python3/dist-packages/xdot/__main__.py", line 22, in <module>
    from .ui.window import DotWindow, Gtk
  File "/usr/lib/python3/dist-packages/xdot/ui/__init__.py", line 11, in <module>
    from .window import DotWidget, DotWindow
  File "/usr/lib/python3/dist-packages/xdot/ui/window.py", line 42, in <module>
    from ..xdotparser import XDotParser
  File "/usr/lib/python3/dist-packages/xdot/ui/_xdotparser.py", line 21, in <module>
    from distutils.version import LooseVersion
ModuleNotFoundError: No module named 'distutils'
```

- vii. Cmd - write_verilog “filename.v”
 1. Creates a netlist file.
 2. use -noattr - gives a cleaner netlist.
- b. Synthesize the design.

“Week 1 Notes”

2. Libraries

- a. “sky130_fd_sc_hd_tt_025C_1v80”.
 - i. tt - typical, [also can be fast or slow]!
 - ii. 025C - [temperature]!
 - iii. 1v80 - [voltage]!
 - iv. **PVT** - determine the behaviour of silicon under these variations.
 - 1. Process - variation due to fabrication.
 - 2. Voltage - variation due to voltage.
 - 3. Temperature - variation due to temperature.
 - v. Libraries are characterized to model these variations.
 - vi. Technology - “cmos”
 - vii. delay_model - “table_lookup”
 - viii. Time_unit - “1ns”
 - ix. V_unit - “1v”
 - x. Leakage_power_unit - “1nW”
 - xi. Current_unit - “1mA”
 - xii. Pulling_resistance_unit - “1kohm”
 - xiii. Capacitive_load_unit - “1.0000000” “pf”
 - xiv. Cells -
 - 1. Different flavours of the same cells.
 - 2. An equivalent verilog model is available.
 - 3. Power the cell will consume during all possible combinations.
 - 4. Power information.
 - 5. Timing information.
 - 6. Area value.
 - 7. yes

“Week 1 Notes”

3. Hierarchical synthesis and Flat synthesis.

a. Hier synth

- i. synth -top "design_name".
- ii. Instead of an OR gate it gave us an “input inverted nand gate”.
 - 1. Acc. to Morgan’s law OR gate = input inverted nand gate.
 - a. Since stacking PMOS is bad, it provides poor mobility.
 - i. To improve mobility here we need a wider cell - transistor.
 - ii. To make it a “**good logical effort**”.
 - 1. Stacking NMOS is better.
 - 2. CMOS we always interpret is using inverters.
- b. Hierarchy is preserved.
 - 1. Modules are preserved.

c. Flat synth

- i. cmd - flatten.
- ii. Write_verilog “design_name”.
- iii. Hierarchy is not preserved.
 - 1. Gives a single file with all the gates implemented.
- iv.

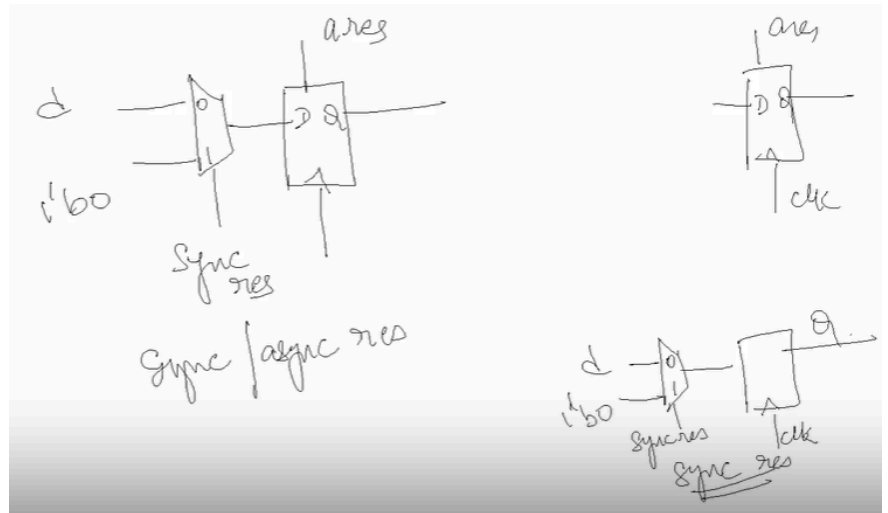
d. Sub-module-lvl-synth

- i. synth -top “sub_module_name”.
 - 1. Why? - to save time!
 - 2. How? - instead of synthesising a module n-times we synth it once and replicate it 6 times and stitch it together in the top module.
 - 3. Also, if we want to follow the “Divide and Conquer” approach. For a large design.
 - 4. And stitch all these netlists at the top level.

“Week 1 Notes”

4. FLOPs!

- a. Why?
 - i. any comb ckt will have some propagation delay.
 - ii. This delay will introduce a “glitch”.
 - iii. More comb ckt more glitches.
 - iv. Output will be glitchy.
 - v. Now we need an element to store the value.
 - vi. FlipFlops are used btw comb ckt.
 - 1. Output only changes with clk edge.
 - 2. Input might glitch but output remains stable.
 - 3. Clk edge shields the output of FF from Input.
 - vii. Flops should be initialised, otherwise the comb inputs will be unknown.
- b. Asyn_res_ff
 - i. +ve clk edge triggered
 - ii. +ve rst edge triggered.
- c. Sync_res_ff
 - i. +ve clk edge triggered
 - ii. Rst only triggers with +ve clk edge.
- d. Async_set_ff
 - i. +ve clk edge triggered
 - ii. Output set to high-1 on rst.
- e. A flop with async and sync reset may cause race_around_condition.



“Week 1 Notes”

- f. yosys - cmd - for FF.
 - i. cmd - dfflibmap -liberty “path of the lib file”.
 - ii. Sometime the standard cell lib and ff lib is different, so we use the two libs here.

“Week 1 Notes”

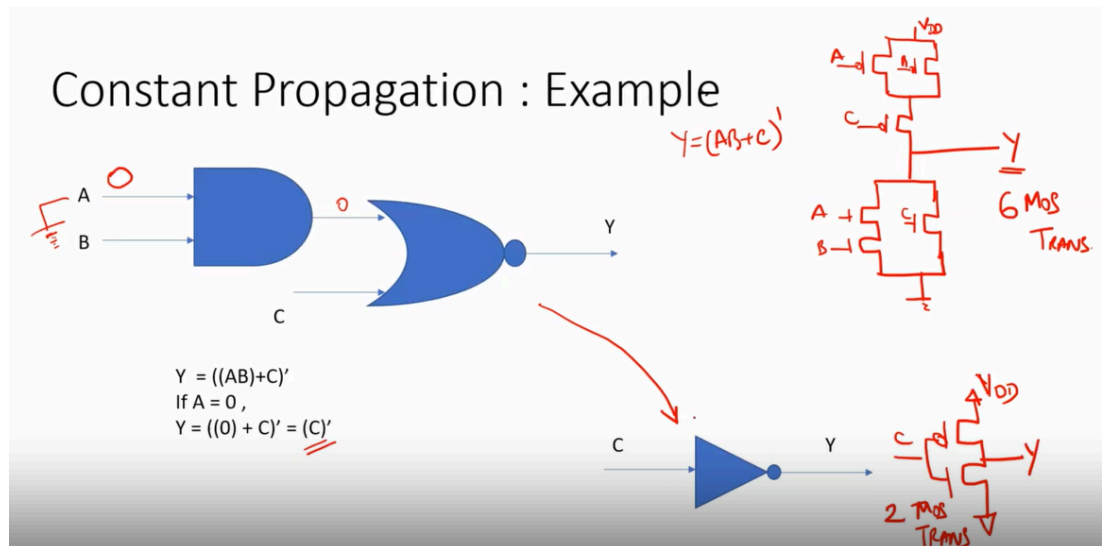
5. Optimizations

a. Logic optimizations

- i. Comb logic
- ii. Seq logic

b. Comb logic optimization

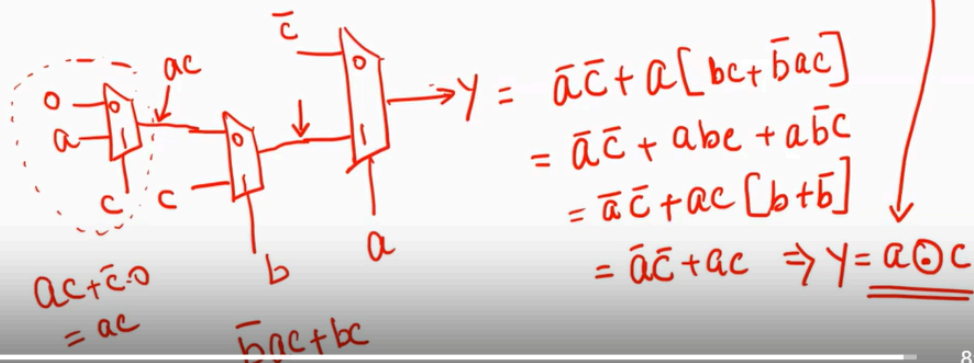
- i. Why? - squeezing the logic to get the most optimized design.
- ii. Result! - reduction in power and area.
- iii. How? -
 1. Constant propagation - Direct optimization.



2. Boolean Logic optimization - using - Kmap | Quine McKluskey

Boolean Logic Optimisation

assign $y = a?(b?c:(c?a:0)):(!c)$



8:06

- a. These optimizations are done by the synthesiser.

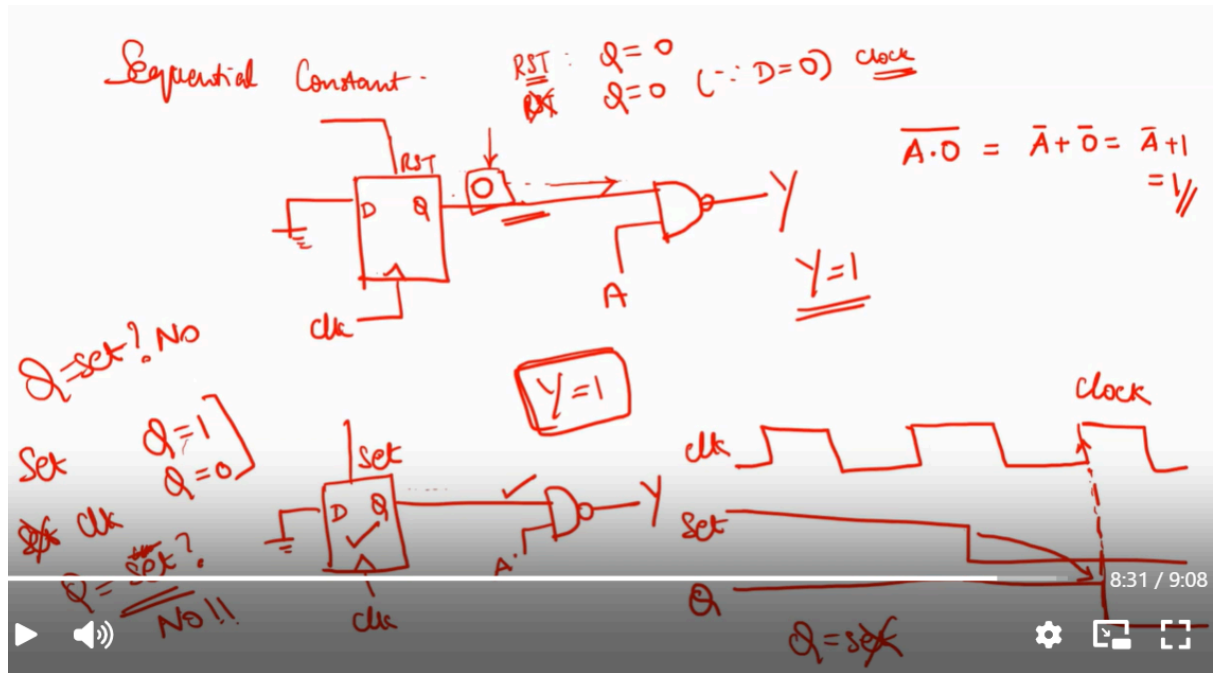
"Week 1 Notes"

c. Sequential Logic Optimizations

i. Basic

1. Sequential constant propagation.

- We have a flop, input tied - low.
- If it is a syn_set FF, it should be retained.



“Week 1 Notes”

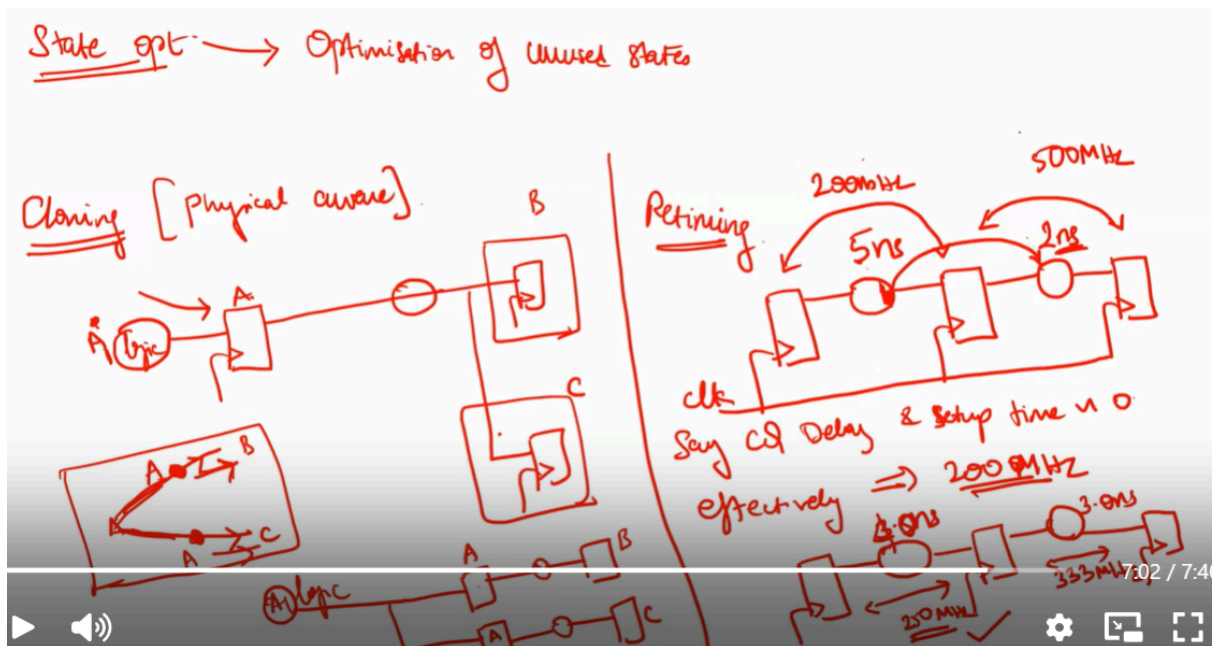
ii. Advanced

1. State Optimisation

- Optimization of unused states - to get the most condensed state machine.

2. Retiming

- Two comb logic btw 3 FF
- FF1oFF2 - 5ns comb delay - clk it at 200MHz
- FF2oFF3 - 2ns comb delay - clk it at 500MHz
- Clk to Q delay and setup time is ~ 0 , so not considered.
- So effective clk freq will be 200MHz at max!
- So now if we partition the comb logic btw these FFs so that we can achieve a higher clk speed greater than 200MHz.
- As we have larger slack at FF3.
- We ate slack from FF3 and reduced the delay of comb logic btw FF1oFF2.
- Used to improve the performance of the circuit.
 - Playing with useful slack for the benefit of performance.



3. Sequential Logic Cloning

- During physical aware synthesis.
- Definition of SLACK!**
 - Slack = **Required Arrival Time** – **Actual Arrival Time**

“Week 1 Notes”

- ii. **Positive Slack (+ve slack)** → The signal arrived **earlier** than the deadline.  Timing is **met** with margin.
- iii. **Zero Slack** → The signal arrived **exactly on time**.  Meets timing but with no margin.
- iv. **Negative Slack (-ve slack)** → The signal arrived **too late**.  Timing violation.
- c. Example: we have a logic connected to a FF and Ou of FF1 is connected to two FF2 and FF3 and FF2 and FF3 are sitting far off - resulting in a large routing delay in FF1 to FF2 and FF1 to FF3.
- d. Let's say we have large +ve slack in our logic.
- e. So we can clone FF1 to FF1oFF2 and FF1oFF3, this will compensate for the large routing delay i.e. timing is made!

d. Sequential optimizations for unused outputs!

- i. Happens during synthesis.
- ii. Unused output optimization.
- iii. Basically output is not internally generated from the module or only some bits are assigned to the output port rest are unused.

“Week 1 Notes”

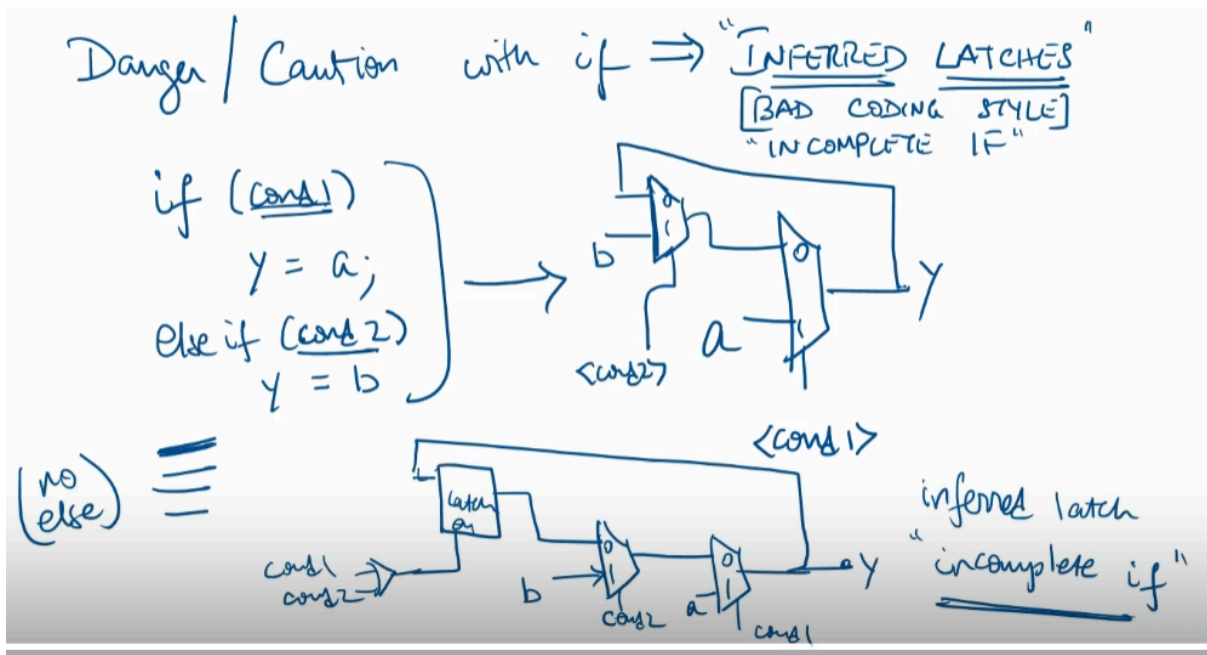
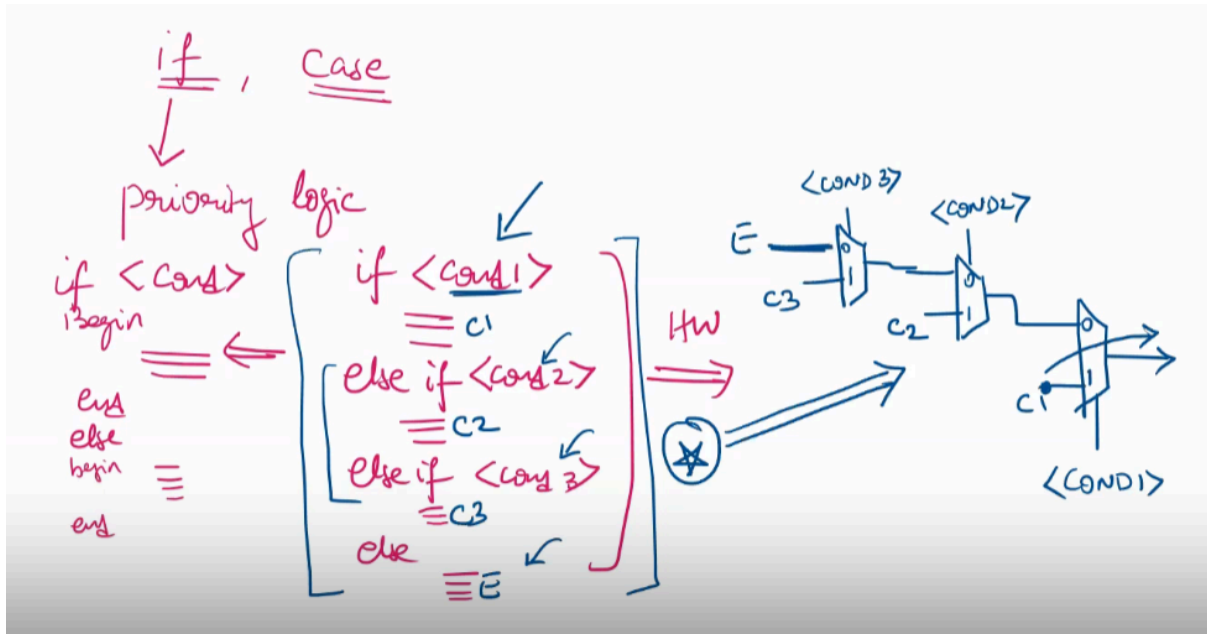
6. Gate Level Simulation - [GLS]!

- a. Running the test bench with the netlist as DUT.
- b. Netlist is logically the same as RTL code.
 - i. The same TB will align with DUT.
- c. Why - GLS ?
 - i. To verify logical correctness of design after synthesis.
 - ii. Ensuring the timing of the design is met.
 - 1. For this GLS needs to be run with delay annotations.
- d. Synthesis and Simulation mis-matches.
 - i. Missing sensitivity list.
 - 1. Can create latches!
 - ii. Blocking vs Non-Blocking Assignments.
 - 1. Serial execution - blocking assignments.- Combinational logic.
 - 2. Parallel execution - non-blocking assignments.- for sequential logic.
 - 3.
 - iii. Non-Standard Verilog Coding.

"Week 1 Notes"

7. Case and IF-else construct

- In comb ckt i cannot have inferred latch.
- In seq ckt it can be there.- ??
- If-els can



"Week 1 Notes"

Counter
 reg [2:0] count;
 always @ (posedge clk, posedge reset)
 begin
 if (reset)
 count <= 3'b000;
 else if (en)
 count <= count + 1;
 end

✓ incomplete if

if no en; count should latch on to the previous value.

Case Statement [if, case are used inside always block]

reg Variable

reg y
 always @ (*).
 begin
 case (sel)
 2'b00 : begin
 c1
 end
 2'b01 : begin
 c2
 end
 2'b10 :
 2'b11 :
 endcase

ans

“Week 1 Notes”

Caveats with Case

① Incomplete Case $\Rightarrow$ Inferred latches $\rightarrow$ CODE CASE WITH DEFAULT

reg [1:0] sel
always @ (x)
begin
 case (sel)
 2'b00: $\equiv$
 c1 $\equiv$
 2'b01: $\equiv$
 c2 $\equiv$
 endcase
end

begin
 case (sel)
 2'b00: $\equiv$
 2'b01: $\equiv$
 default: $\equiv$
 endcase
end

with default $\Rightarrow$ we avoid inferred latches. (X)

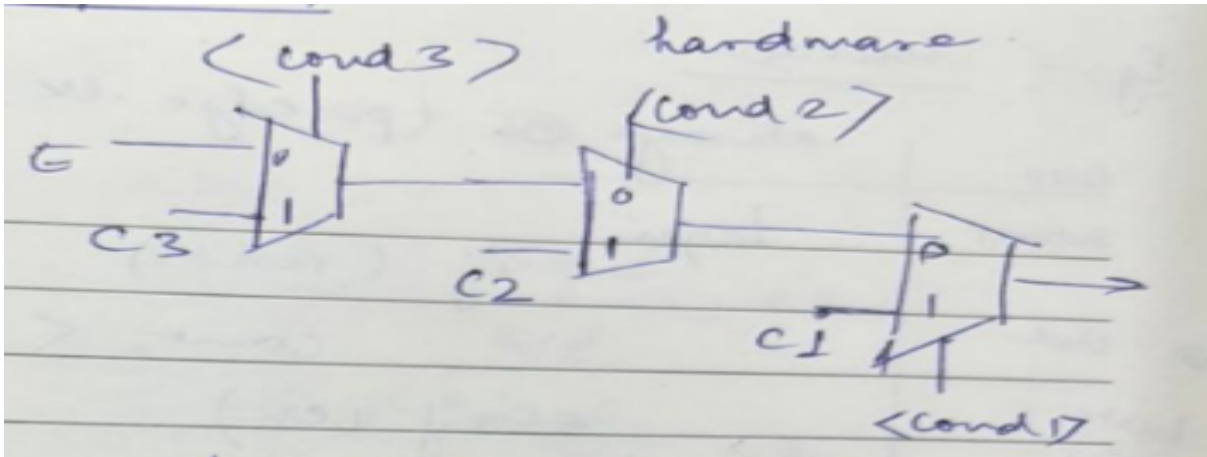
If statement :

```

if (cond1)
    C1
else if (cond2)
    C2
else if (cond3)
    C3
else
    E
end

```

"Week 1 Notes"



Danger/Caution with if

→ Inferred latches

Due to bad coding style, latches can come with incomplete if-conditions.

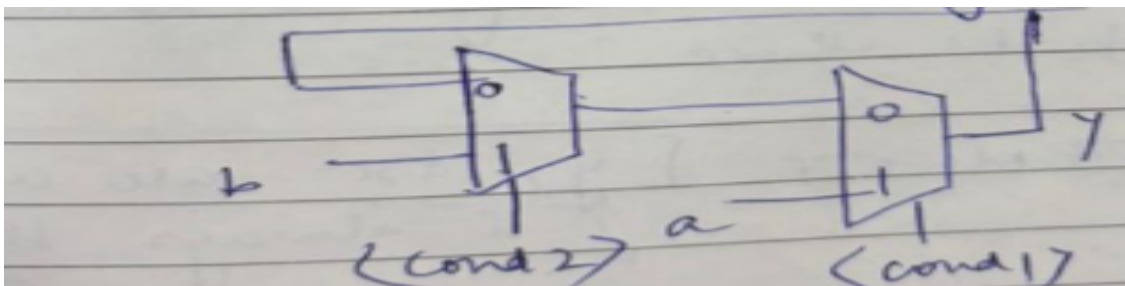
if (cond1)

 y = a;

else if (cond2)

 y = b;

// no else statement



"Week 1 Notes"

Hardware Representation If no else, tool cannot determine what happens when cond1 and cond2 are false → combinational loop or latch may form.

Correct Way (to avoid latch inference)

Add else to cover all cases. Example:

if (cond1)

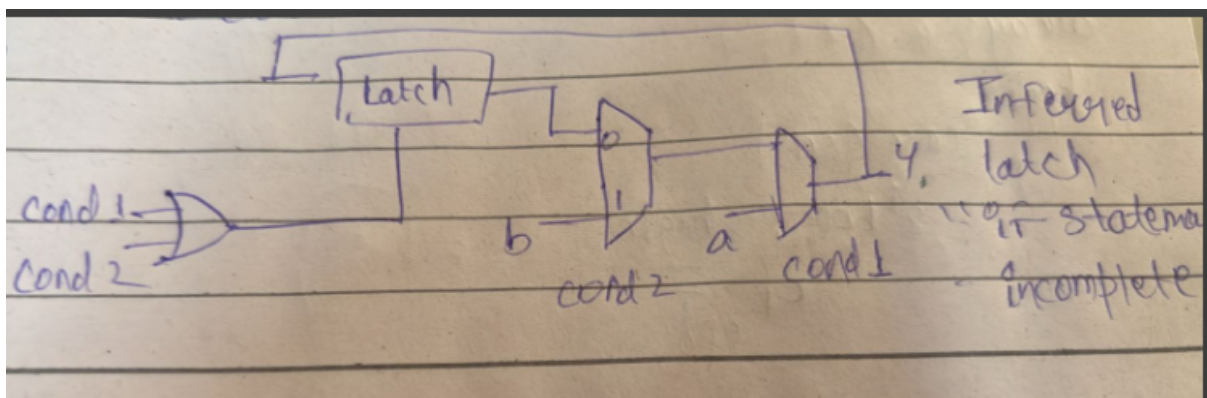
 y = a;

else if (cond2)

 y = b;

else

 y = c;



Example: Counter

always @ (posedge clk, posedge reset)

begin

 if (reset)

 count <= 3'b000;

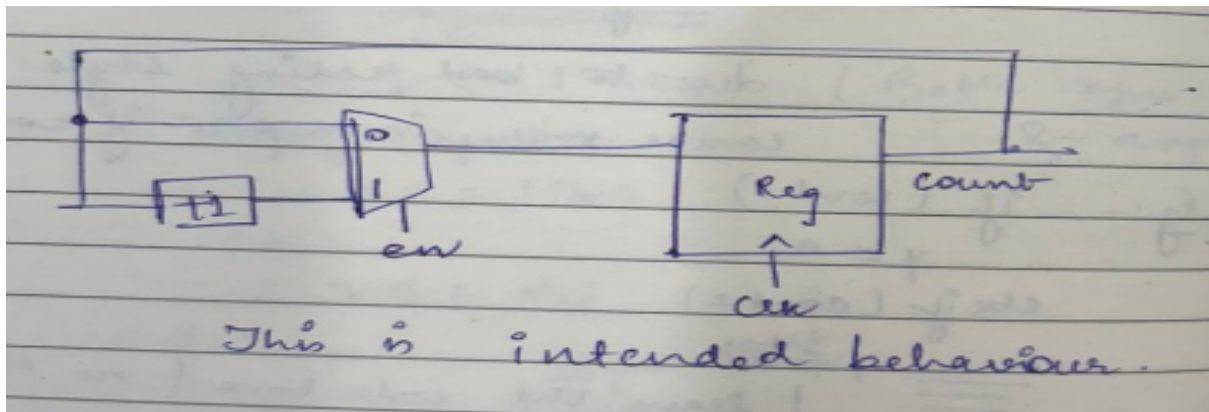
 else if (en)

 count <= count + 1;

end

(no else — but still this code is fine)

"Week 1 Notes"



Case Statement

If **case** is used in **always block**, variable used should be of **reg type**.

```
reg Y;
```

```
always @ (*)
```

```
begin
```

```
  case (sel)
```

```
    2'b00: begin
```

```
      Y = C1;
```

```
    end
```

```
    2'b01: begin
```

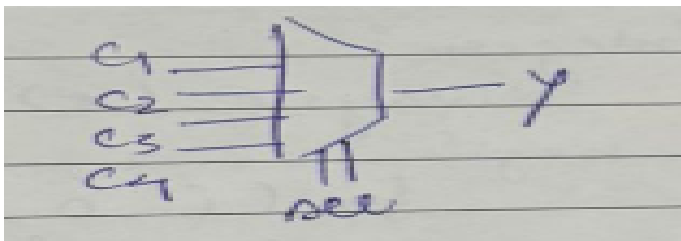
```
      Y = C2;
```

```
    end
```

```
    // ... other cases
```

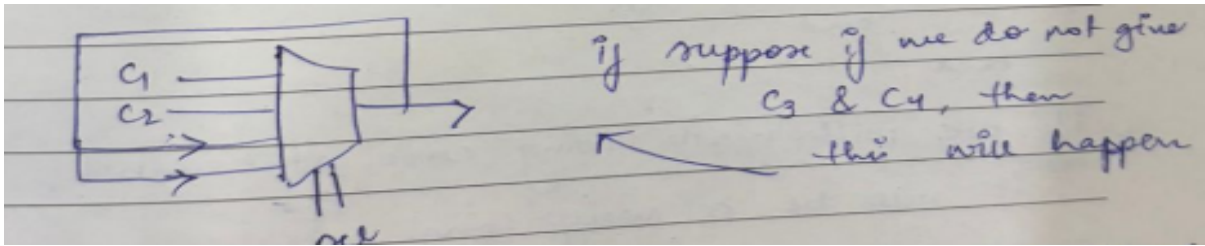
```
  endcase
```

```
end
```



⚠ Incomplete case statement leads to inferred latches

If we only code for some cases (say only C1, C2) but do not handle C3 & C4, then synthesis will infer a latch.



✓ Solution:

Always **code case with default**

default: Y = something;

With this, we avoid inferred latches.

LOOPING CONSTRUCTS

1. for loop
2. generate followed by for loop

- **Used inside always** → (for loop)
- **Used outside always** → (generate-for loop)
 - Cannot be used inside always

for instantiating hardware → generate-for

for evaluating expressions → normal for loop

Example

Assumption: inp[31:0] → bus

32X integer

```
always @(*) begin
```

```
  for (i = 0; i < 32; i = i + 1) begin
```

```
    if (i == sel)
```

```
      y = inp[i];
```

```
  end
```

```
end
```

If we write using case statements, it will be a very long code.

Demux Example

1×8 Demux

```
always @(*) begin
```

```
  op_bus[7:0] = 8'b0;
```

```
  for (i = 0; i < 8; i = i + 1) begin
```

```
    if (i == sel)
```

```
      op_bus[i] = input;
```

```
  end
```

“Week 1 Notes”

end

For a very wide mux or de-mux, **for loop is very handy.**

for-generate → it is for replicating the hardware

Example:

```
and u_and1 (.a1), .b1), .y(1));
```

```
and u_and2 (.a1), .b1), .y(1));
```

...

Instead of writing multiple lines manually, use **generate** to replicate the hardware.

Using generate to replicate the hardware

```
genvar i;
```

```
generate
```

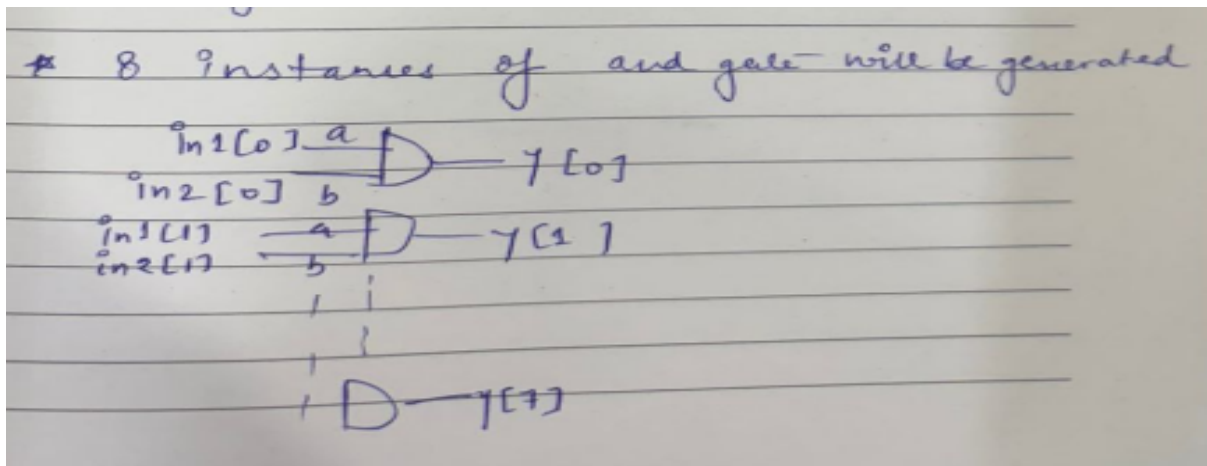
```
  for (i = 0; i < 8; i = i + 1) begin
```

```
    and u_and (.a(in1[i]), .b(in2[i]), .y(y[i]));
```

```
  end
```

```
endgenerate
```

➡ 8 instances of AND gates will be generated automatically.



Need of Replication

Example: Ripple carry adder — needs instantiation of full adders.

If generate can also be used.